

1 The ladder



**Crux.** You never climb past a rung you have not done, and most wrong answers are somebody starting at rung five because they arrived with a theory.

2 What each rung needs

| Rung           | Needs                        | Goes wrong when               |
|----------------|------------------------------|-------------------------------|
| Confirm        | Both totals, one definition  | Delivered meets booked        |
| Like with like | Equal windows, same segments | 13 weeks meets 11             |
| Decompose      | Customers, orders, revenue   | The total is called a finding |
| Isolate        | The same split per segment   | A blend hides the segment     |
| Hypothesise    | Something outside the data   | A cause is stated as fact     |

3 Grouping by key

```
totals = {}
for order in ORDERS:
    k = order["quarter"]
    totals[k] = totals.get(k, 0) + order["amount"]
```

Count adds one. Revenue adds the amount. Distinct customers uses `setdefault(k, set()).add(...)`.

4 The decomposition has to close

REVENUE = CUSTOMERS x ORDERS PER CUSTOMER x REVENUE PER ORDER

| Factor              | Q1     | Q2     | Change     |
|---------------------|--------|--------|------------|
| Customers           | 69     | 69     | flat       |
| Orders per customer | 1.65   | 1.25   | down 24.6% |
| Revenue per order   | 1.84L  | 2.17L  | up 18.0%   |
| Revenue             | 2.10cr | 1.87cr | down 11.0% |

**Crux.** Multiply the three factor changes and they must land on the revenue change, or a factor is missing and nothing built on the decomposition is safe.

5 return, never print

|               |                                                          |
|---------------|----------------------------------------------------------|
| <b>return</b> | <b>Hands the value back, so the next line can use it</b> |
| <b>print</b>  | Draws on a screen and hands back <code>None</code>       |

A function that prints cannot be built on. The tell is `TypeError: 'NoneType' object is not subscriptable` one line later.

6 Defaults are decisions

`KeyError: 'discount'` means the key is not there. Before reaching for `.get()`, count how many records and check whether the absence is concentrated.

| Choice          | What it costs                           |
|-----------------|-----------------------------------------|
| Drop the row    | Revenue falls and the count changes     |
| Default to zero | The count holds, revenue is understated |
| Keep and flag   | Nothing lost, somebody has to look      |

**Crux.** There is no free option, so write the reason beside the default, because on Wednesday somebody from Finance asks for it.

7 A finding, not an opinion

A cause with a test beside it is a finding. A cause without one is an opinion with numbers attached.

Retail-Plus down 49 percent against Retail-Core's 5 is a finding. "The reorder button broke" is a hypothesis until reorder events per member either side of the window say so.

THE WORDS ON THIS SHEET

- Investigation ladder** The fixed order in which a drop is investigated
- Decomposition** Revenue rewritten as factors whose changes multiply to the whole change
- Accumulator per key** A dictionary holding one running total for each group
- Hypothesis** A stated cause with the evidence that would settle it beside it
- Like with like** Two periods comparable on window, definitions and denominators
- Segment** A customer grouping the business already uses, here four of them
- Default** The value returned when a key is absent, and a decision somebody has to...
- Return** Handing a value back to the caller, as against printing it to a screen